

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250131>

Outline : Decision Trees

- Definition
- Training and Visualizing a Decision Tree
- Making Predictions
- Estimating Class Probabilities
- The CART Training Algorithm
- Computational Complexity
- Gini Impurity or Entropy?
- Regularization Hyperparameters
- Regression
- Instability

—

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapter 6**

Definition (1)

Like SVMs, *Decision Trees* are versatile Machine Learning algorithms: classification and regression tasks, also multioutput tasks.

Fundamental for Random Forests (next week).

A decision tree is a flowchart-like description of a map $h : \mathbf{X} \rightarrow \mathbf{y}$ which maps the features $x \in \mathbf{X}$ of a datapoint to a predicted label $h(x) \in \mathbf{Y}$

Definition (2)

A decision tree is a flowchart-like description of a map $h : \mathbf{X} \rightarrow \mathbf{Y}$ which maps the features $\mathbf{x} \in \mathbf{X}$ of a datapoint to a predicted label $h(\mathbf{x}) \in \mathbf{Y}$

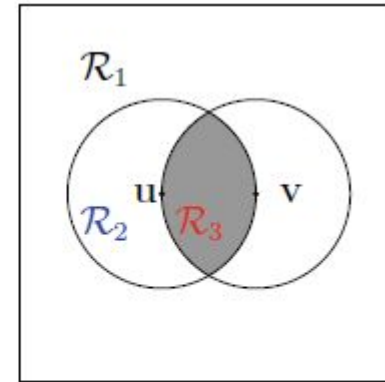
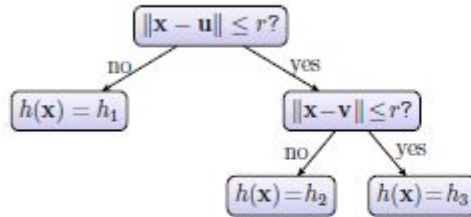


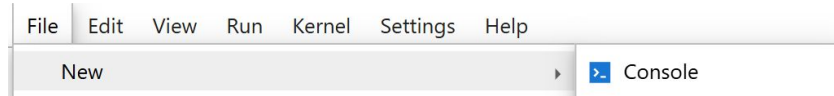
Fig. 3.8 A decision tree represents a hypothesis h which is constant on subsets $\mathcal{R}_m$, i.e., $h(\mathbf{x}) = h_m$ for all $\mathbf{x} \in \mathcal{R}_m$. Each subset $\mathcal{R}_m \subseteq \mathcal{X}$ corresponds to a leaf node in the decision tree

Training and Visualizing a Decision Tree

Steps 1-3

Configuring Graphviz

1. Installing the Graphviz : **pip install graphviz** in console



2. To set / add the path to the graphviz executables in the Notebook. Check the actual path for YOUR computer.

```
import os  
os.environ["PATH"] += os.pathsep + 'C:/Program Files (x86)/Graphviz2.38/bin/'
```

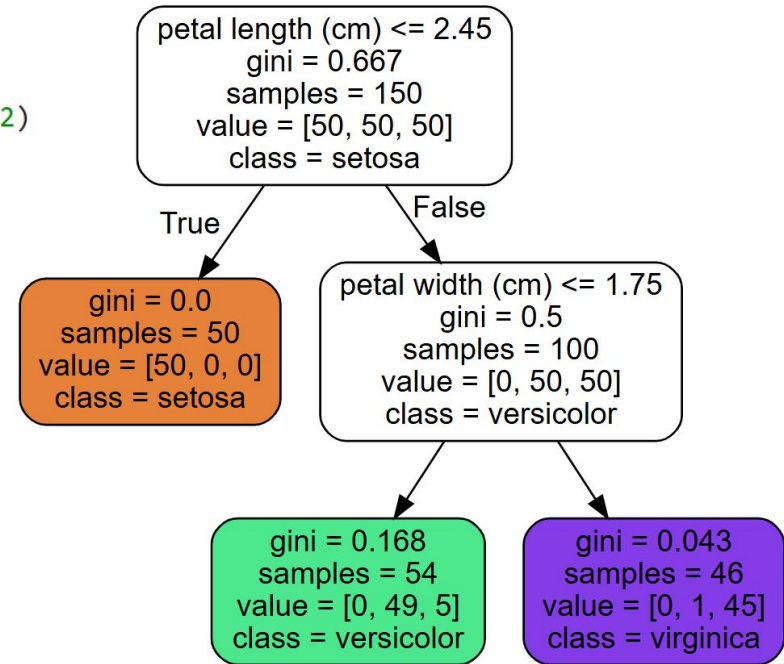
[Stackoverflow](#) : Setting up the path

(Side reading: [Graphviz](#) in Jupyter Notebook)

Training and Visualizing a Decision Tree (1)

DecisionTreeClassifier

```
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)  
tree_clf.fit(X, y)
```



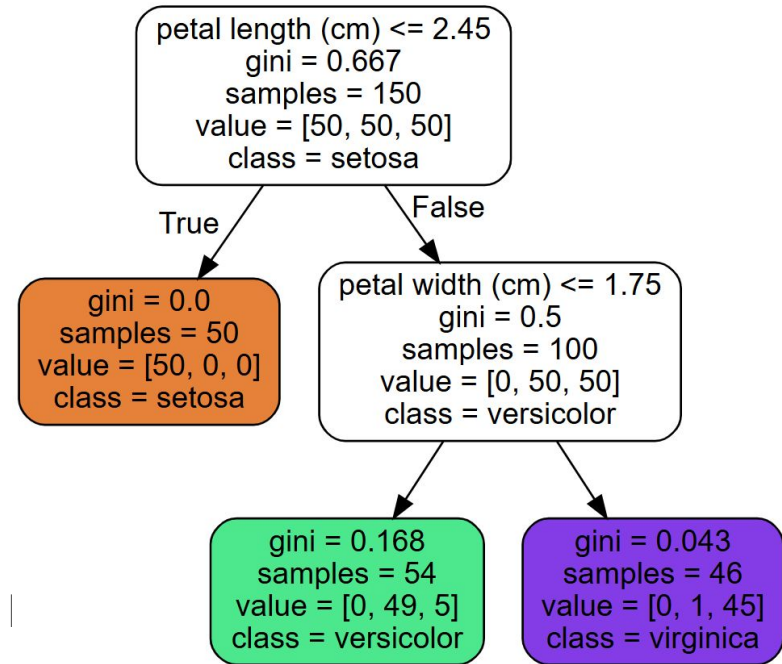
Making predictions

Step 4

Making predictions (1)

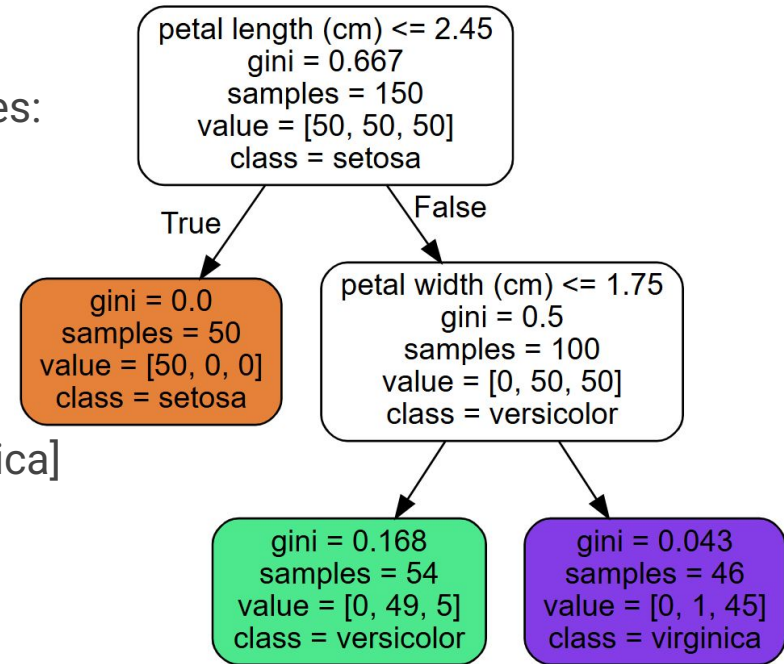
1. Starting *root node* (depth 0).
petal length ≤ 2.45
2. **Yes:** (depth 1, left) *leaf node* $\rightarrow$ setosa
3. **No:** (depth 1, right)
petal width ≤ 1.75
4. **Yes:** (depth 2, left) *leaf node* $\rightarrow$ versicolor
5. **No:** (depth 2, right) *leaf node* $\rightarrow$ virginica

(Decision Trees require very little data preparation. |
feature scaling or centering at all.)



Making predictions (2)

- samples:** counts how many training instances.
 Example: (depth 1, right) → **100** training instances:
 (depth 2, left) **54** (petal width ≤ 1.75).
 (Note: $54 + 46 = 100$ - depth 2 nodes)
- value:** how many training instances of each class this node applies to.
 Example: (depth 2, right)
 [0 Iris-Setosa, 1 Iris- Versicolor, and 45 Iris-Virginica]
- gini:** Node *impurity* - “pure” (gini=0)



Making predictions (3)

- gini:** Node *impurity* - “pure” (gini=0)

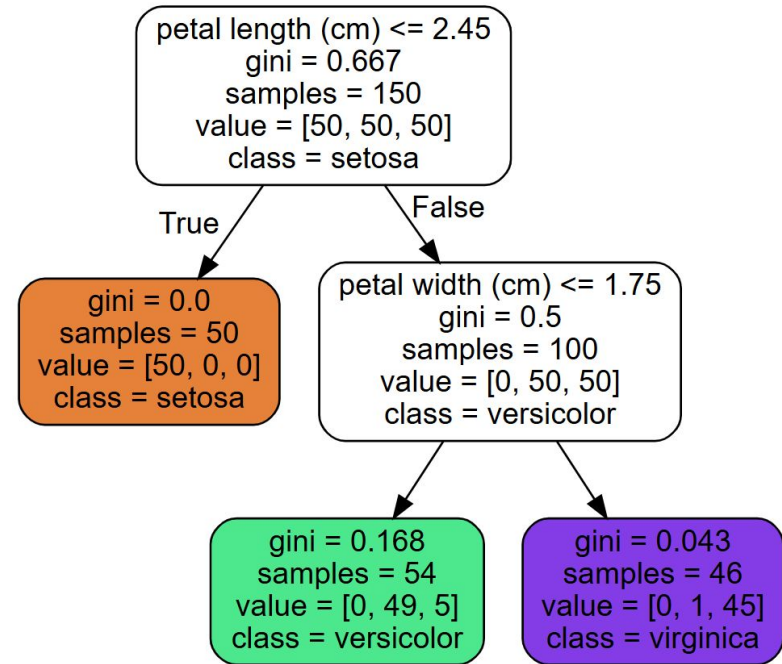
Equation 6-1. Gini impurity

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

, where $p_{i,k}$ is the ratio of class k instances among the training instances in the i^{th} node.

Example: (depth 2, left)

$$1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0.168$$



Making predictions (4)

Decision tree boundaries. (depth 0): petal length = 2.45 cm. (Iris-Setosa to the left - pure area). "Right area" - impure. So, (depth 1): petal width = 1.75 cm.

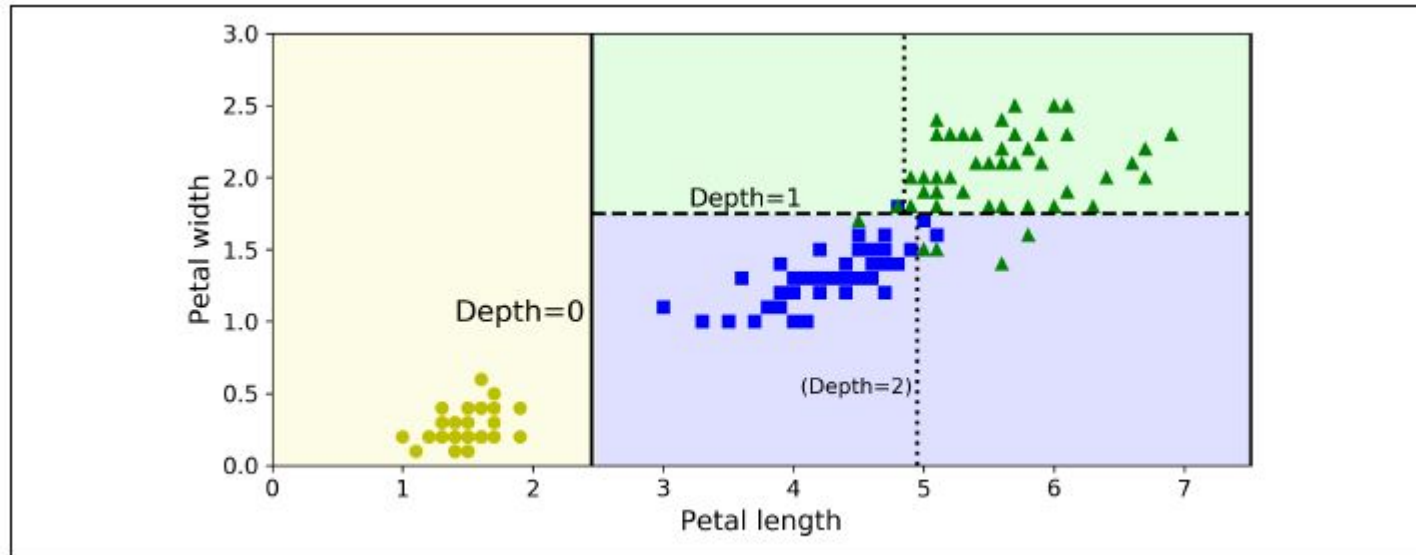


Figure 6-2. Decision Tree decision boundaries

Making predictions (5) : Explainability

Decision trees : decisions are easy to interpret. *White box models*.

Random Forests or ANN: *black box methods* - great predictions, but hard to explain why these predictions.

Estimating Class Probabilities

Step 5

Estimating Class Probabilities

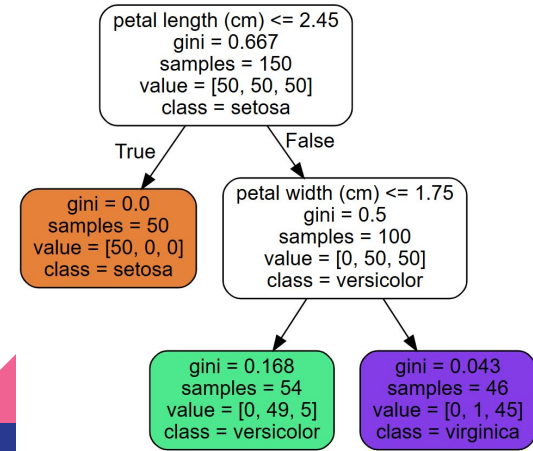
- Decision Trees can estimate probabilities.
- An instance belongs to a particular class k : first it traverses the tree to find the leaf node for this instance, and then it returns the ratio of training instances of class k in this node.

Example: (depth 2, left) $\rightarrow$ 0% for Iris-Setosa (0/54), 90.7% for Iris-Versicolor (49/54), and 9.3% for Iris-Virginica (5/54).

Try: petal length = 5, petal width = 1.5

```
>>> tree_clf.predict_proba([[5, 1.5]])
array([[0.          , 0.90740741, 0.09259259]])
```

```
>>> tree_clf.predict([[5, 1.5]])
array([1])
```



The CART Training Algorithm

Classification and Regression Tree

The CART Training Algorithm (1)

Classification And Regression Tree (CART) algorithm to train / grow Decision Trees.

- First splits the training set in two subsets using a single feature k and a threshold t_k (e.g., “petal length ≤ 2.45 cm”).
 - How does it choose k and t_k ? It searches for the pair (k, t_k) that produces **the purest subsets** (weighted by their size).

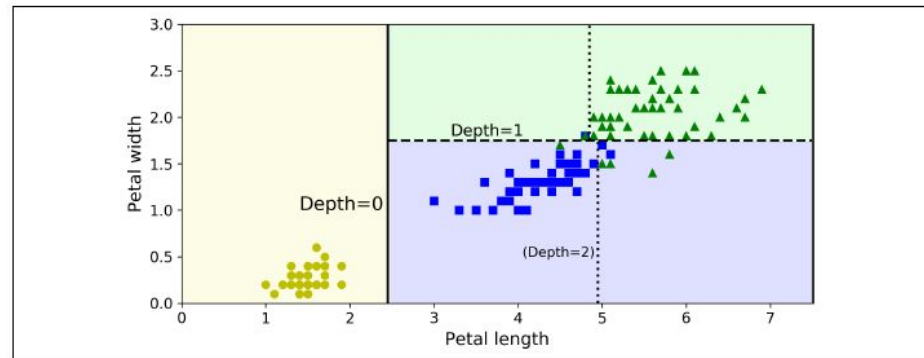


Figure 6-2. Decision Tree decision boundaries

The CART Training Algorithm (2)

To minimize

Equation 6-2. CART cost function for classification

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

where $\begin{cases} G_{\text{left/right}} \text{ measures the impurity of the left/right subset,} \\ m_{\text{left/right}} \text{ is the number of instances in the left/right subset.} \end{cases}$

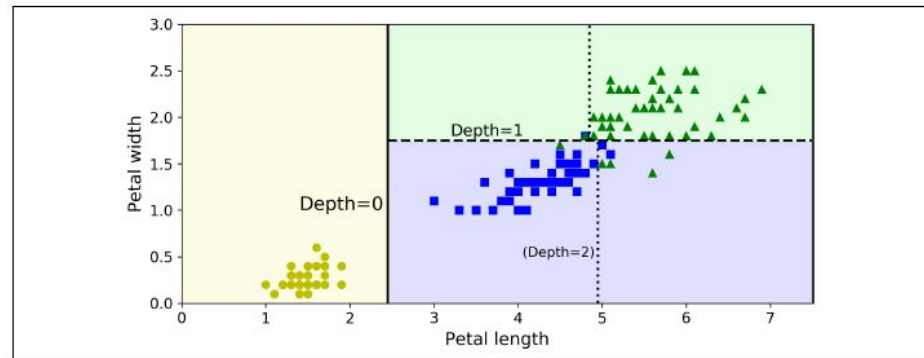


Figure 6-2. Decision Tree decision boundaries

The CART Training Algorithm (3)

- Once it has successfully split the training set in two, it splits the subsets using the same logic, then the sub-subsets and so on, recursively.
- It stops recursing once it reaches the maximum depth (defined by the **max_depth** hyperparameter), or if it cannot find a split that will reduce impurity.

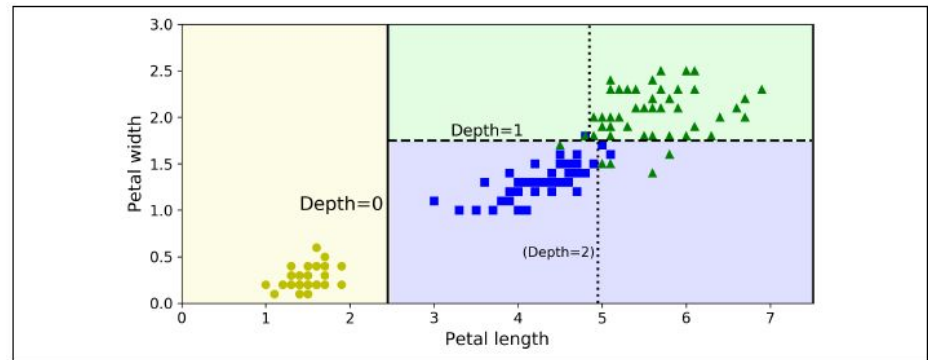


Figure 6-2. Decision Tree decision boundaries

The CART Training Algorithm (4)

DecisionTreeClassifier

```
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)  
tree_clf.fit(X, y)
```

Can be also fine tuned with:

min_samples_split, min_samples_leaf, min_weight_fraction_leaf, and max_leaf_nodes.

Computational Complexity

Computational Complexity

- Making predictions requires traversing the Decision Tree from the root to a leaf.
- Decision Trees are generally approximately balanced, so traversing the Decision Tree requires going through roughly $O(\log_2(m))$ nodes. Checking a value of one feature at each node. → **Predictions are very fast.**
- However, training algorithm compares all features (or less if **max_features** is set) on all samples at each node. This results in a training complexity of $O(n \times m \log(m))$.



Gini Impurity or Entropy?

Gini Impurity or Entropy? (1)

- Gini impurity measure by default.
- However, also the *entropy* impurity measure is possible by setting the criterion hyperparameter to "**entropy**".
- Entropy: a measure of molecular disorder (entropy approaches zero when molecules are still and well ordered).
- Shannon's information theory: measures the average information content of a message (entropy is zero when all messages are identical).
- In ML, it is frequently used as an impurity measure: a **set's entropy is zero** when **it contains instances of only one class**.

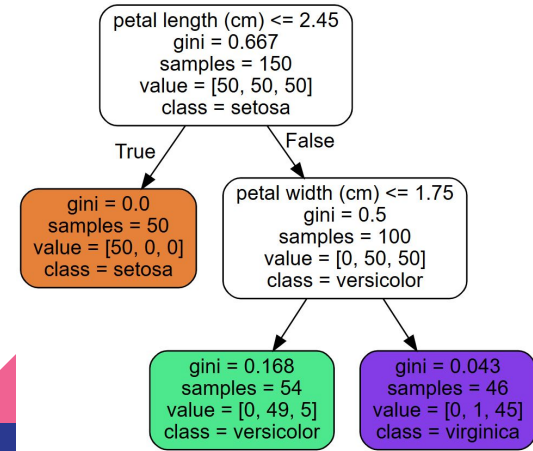
Gini Impurity or Entropy? (2)

Equation 6-3. Entropy

$$H_i = - \sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^n p_{i,k} \log_2 (p_{i,k})$$

Example: (depth 2, left)

$$-\frac{49}{54} \log_2 \left(\frac{49}{54} \right) - \frac{5}{54} \log_2 \left(\frac{5}{54} \right) \approx 0.445$$



Gini Impurity or Entropy? (3)

Gini impurity or entropy? Most of the time it does not make a big difference: they lead to similar trees.

Gini impurity is slightly faster to compute, so it is a good default.

However, when they differ, Gini impurity tends to isolate the most frequent class in its own branch of the tree, while entropy tends to produce slightly more balanced trees.

Regularization Hyperparameters

Steps 7-11

Regularization Hyperparameters (1)

- To avoid overfitting the training data: to restrict the Decision Tree's freedom during training. → Regularization.
- The regularization hyperparameters depend on the algorithm used.
- At least restrict the maximum depth of the Decision Tree.
 - **max_depth** hyperparameter (the default value is None, which means unlimited). Reducing max_depth will regularize the model and thus reduce the risk of overfitting.

Regularization Hyperparameters (2)

The DecisionTreeClassifier class has a few other parameters that similarly restrict the shape of the Decision Tree:

- **min_samples_split** : the minimum number of samples a node must have before it can be split,
- **min_samples_leaf** : the minimum number of samples a leaf node must have,
- **min_weight_fraction_leaf** : same as min_samples_leaf but expressed as a fraction of the total number of weighted instances,
- **max_leaf_nodes** : maximum number of leaf nodes,
- **max_features** : maximum number of features that are evaluated for splitting at each node.

Increasing min_* hyperparameters or reducing max_* hyperparameters will regularize the model.

Regularization Hyperparameters (3)

`min_samples_leaf=4.`

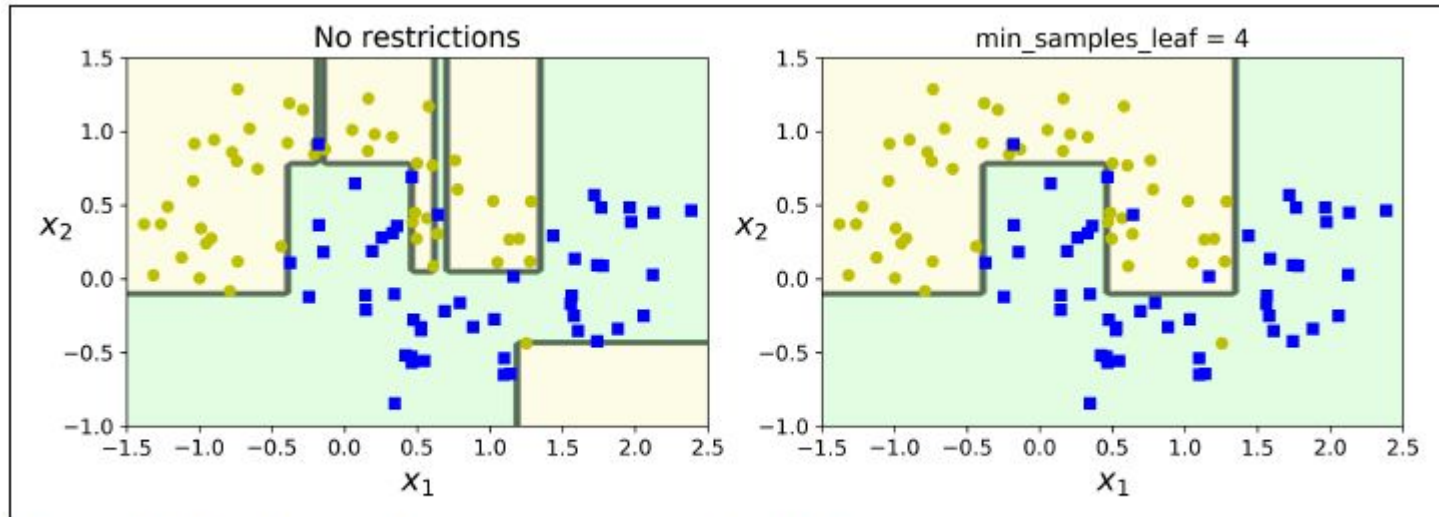


Figure 6-3. Regularization using `min_samples_leaf`

Regression

Steps 12-17

Regression (1)

DecisionTreeRegressor

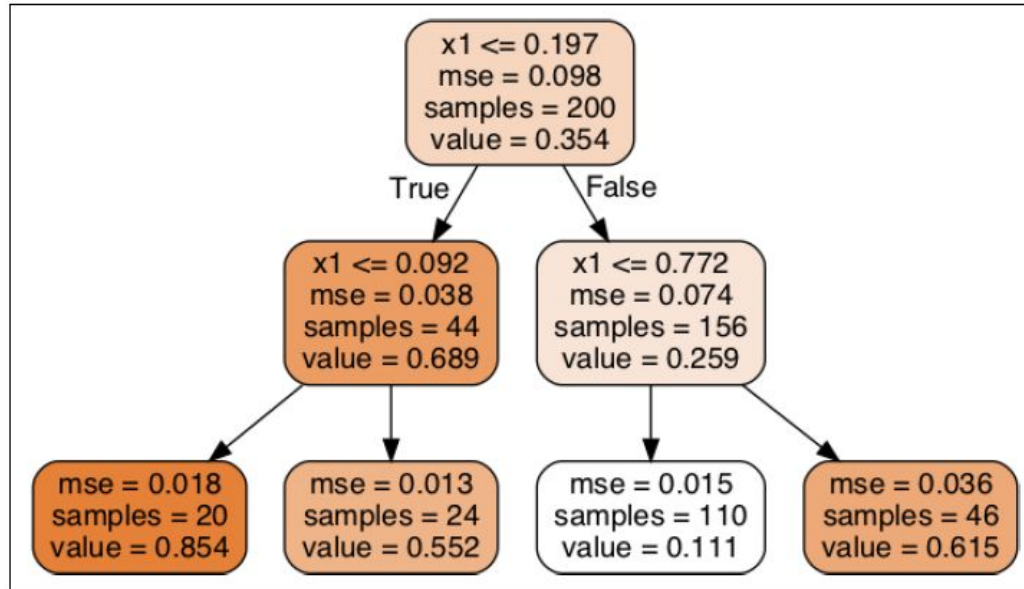


Figure 6-4. A Decision Tree for regression

Regression (2)

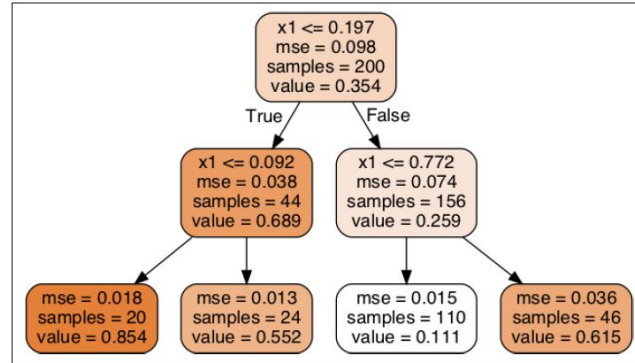


Figure 6-4. Decision Tree Structure

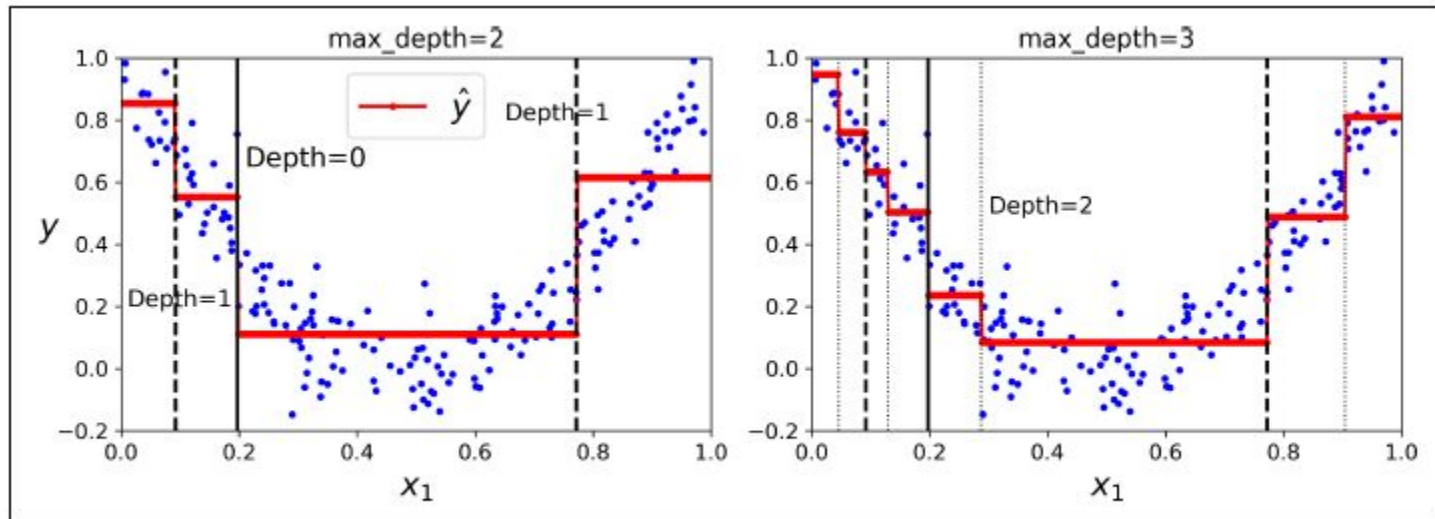


Figure 6-5. Predictions of two Decision Tree regression models

Regression (3) : CART cost function

Equation 6-4. CART cost function for regression

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} \quad \text{where} \quad \begin{cases} \text{MSE}_{\text{node}} = \sum_{i \in \text{node}} (\hat{y}_{\text{node}} - y^{(i)})^2 \\ \hat{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)} \end{cases}$$

Regression (4)

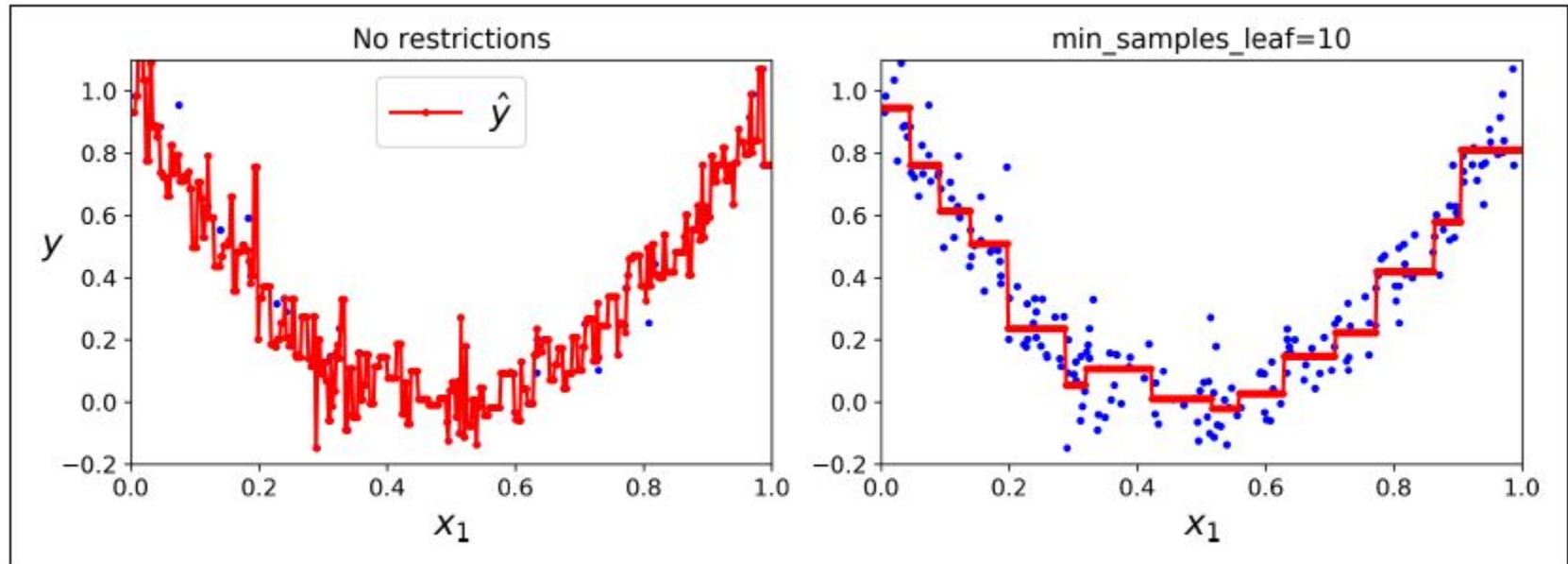


Figure 6-6. Regularizing a Decision Tree regressor

Instability

Step 8, 10, 11

Instability (1) : Sensitivity to training set rotation

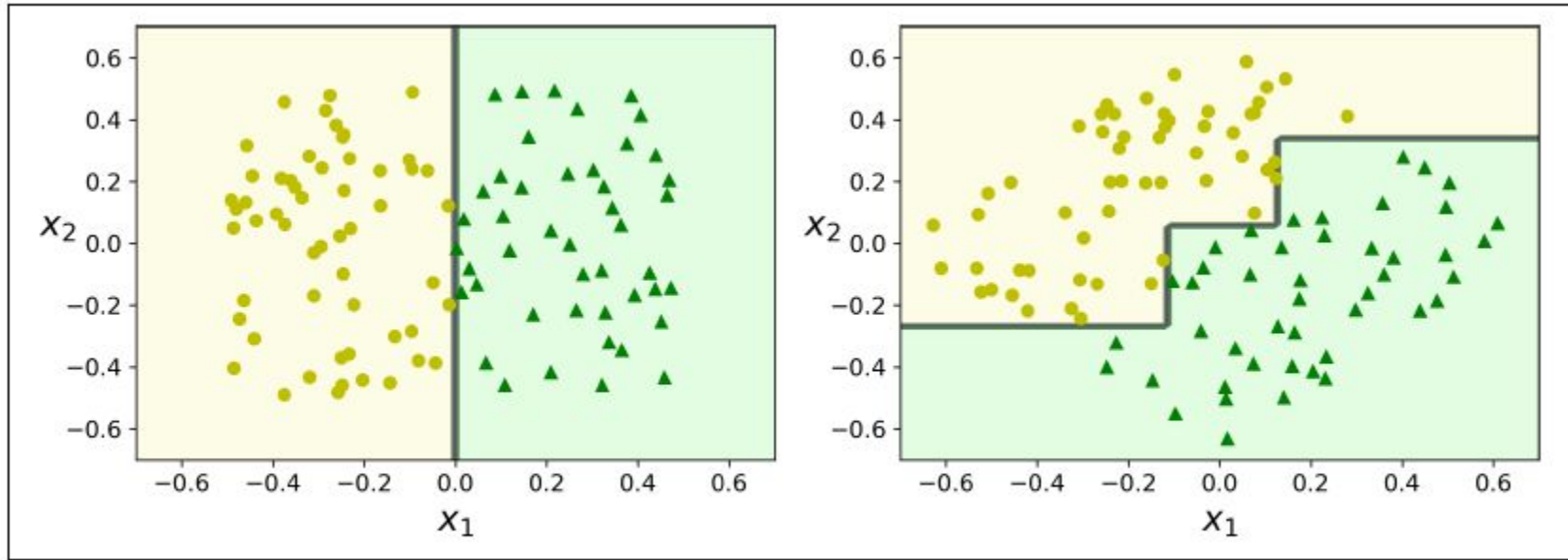
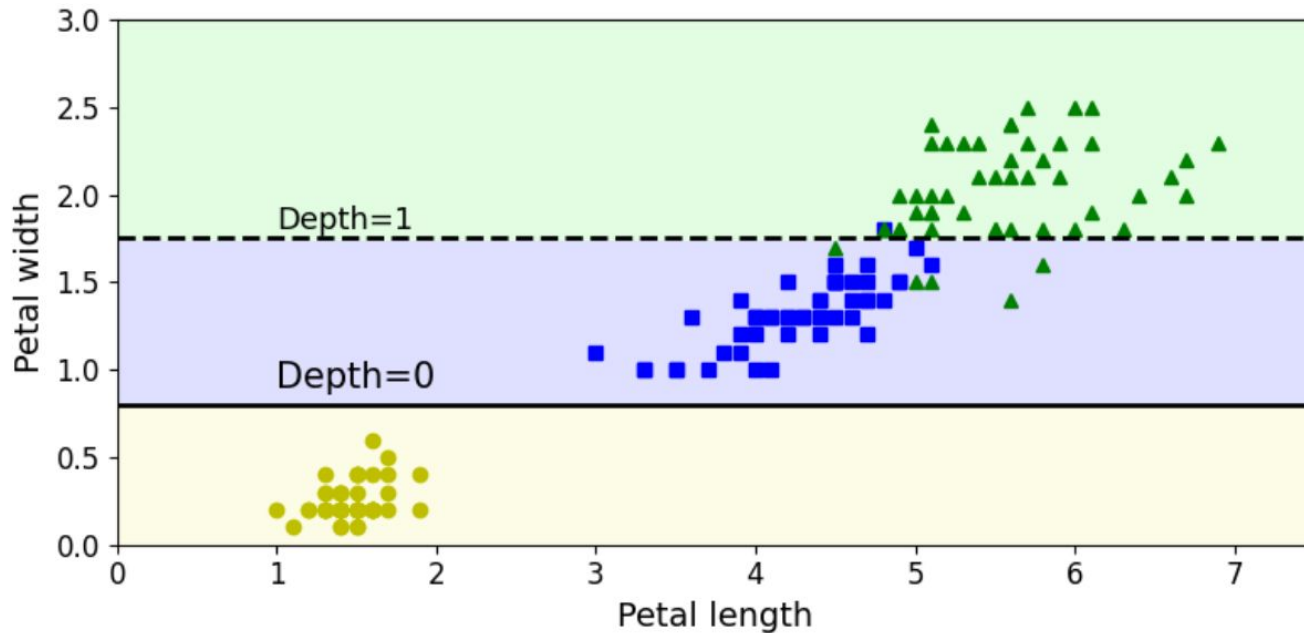


Figure 6-7. Sensitivity to training set rotation

Instability (2) : Sensitivity to training set details

random_state=40.



Takeaways

- Definition, implementation and “navigation” of Decision Trees.
- Can be applied for classification and regression tasks. A white box model.
- Hyperparameters of DecisionTreeClassifier and DecisionTreeRegressor and their affect.